HTML/CSS

Q1.

SNIPPET

What is the output / behaviour of the following HTML snippet?

```
<div style="display:flex; justify-content:center; align-items:center;
height:100px;">
<p>Hello</p>
</div>
```

A) 'Hello' is aligned to the top-left of the div
B) 'Hello' is horizontally and vertically centred inside the 100 px-tall div
C) 'Hello' overflows the div because flex is not a valid value
D) 'Hello' disappears because height is fixed

Answer: B display:flex turns the div into a flex container. justify-content:center centres children along the main axis (horizontal by default) and align-items:center centres them along the cross axis (vertical). So 'Hello' sits dead-centre in the 100 px box.

Q2.

MCQ

Which CSS property controls the space BETWEEN the content and the border of an element?

A) margin
B) padding
C) border-spacing
D) gap

Answer: B padding is the space between the element's content and its border. margin is the space OUTSIDE the border. border-spacing is only for table cells, and gap is used in flex/grid layouts.

Q3.

What colour will the paragraph text appear?

```
<style>
p { color: blue; }
.highlight { color: red; }
#main { color: green; }
</style>
<p class="highlight" id="main">Hello</p>
```

A) Blue — element selector wins
B) Red — class selector wins
C) Green — ID selector wins

SNIPPET

D) Black — conflicts cancel out

Answer: C CSS specificity order: ID (0,1,0,0) > class (0,0,1,0) > element (0,0,0,1). The #main rule has the highest specificity, so the text is green.

MCQ

Q4.

What does the HTML5 tag convey?

A) A navigation menu
B) A self-contained piece of content that could stand alone (e.g. a blog post)
C) A sidebar widget
D) A highlighted quotation

Answer: B represents an independent, self-contained piece of content meant to be reusable — like a news article, blog post, or forum post. is for navigation, for sidebars, and for quotations.

Q5.

What is the computed width of the .box element in a standard (content-box) box model?

```
.box {
width: 200px;
padding: 20px;
border: 5px solid black;
margin: 10px;
box-sizing: content-box;
}
```

A) 200px
B) 250px
C) 260px
D) 270px

Answer: B With box-sizing:content-box (the default), width sets only the CONTENT area. Total rendered width = content(200) + left padding(20) + right padding(20) + left border(5) + right border(5) = 250px. Margin is outside the box and not included.

JavaScript

SNIPPET

Q6.

What does the following code print to the console?

```
console.log(typeof null);
```

A) 'null'
B) 'undefined'
C) 'object'
D) 'number'

SNIPPET

Answer: C This is a famous JavaScript bug that has existed since its first version. typeof null returns 'object' even though null is not an object. It is preserved for backward compatibility.

Q7.

What is logged?

```
let x = 10;
function foo() {
console.log(x);
let x = 20;
}
foo();
```

A) 10
B) 20
C) undefined
D) ReferenceError

Answer: D Variables declared with let are NOT hoisted to the top of the function in an initialised state. They sit in a 'Temporal Dead Zone' (TDZ) from the start of the block until the declaration line. Accessing x before let x = 20 throws a ReferenceError.

Q8.

SNIPPET

What is the output?

```
console.log(1 + '2' + 3);
```

A) 6
B) '123'
C) '15'
D) NaN

Answer: B JavaScript evaluates left-to-right. 1 + '2' triggers string concatenation → '12'. '12' + 3 also triggers concatenation → '123'. The result is the string '123'.

Q9.

MCQ

Which of the following correctly describes the difference between == and === in JavaScript?

A) == compares only types; === compares only values
B) == checks value after type coercion; === checks value AND type without coercion

C) They are identical in all modern browsers

D) === is used only for objects

SNIPPET

Answer: B == (loose equality) performs type coercion before comparing — so '5' == 5 is true. === (strict equality) compares both value and type without coercion — so '5' === 5 is false. Always prefer === to avoid unexpected bugs.

Q10.

What is logged?

```
const arr = [1, 2, 3];
const doubled = arr.map(n => n * 2);
console.log(doubled);
```

A) [1, 2, 3]
B) [2, 4, 6]
C) undefined
D) [1, 4, 9]

Answer: B Array.prototype.map() returns a NEW array where each element is the result of calling the callback. Here each element n is multiplied by 2, producing [2, 4, 6]. The original array is unchanged.

Q11.

SNIPPET

What is the output?

```
var a = 5;
function test() {
var a = 10;
console.log(a);
}
test();
console.log(a);
```

A) 10, 10
B) 10, 5
C) 5, 10
D) 5, 5

Answer: B var is function-scoped. Inside test(), var a = 10 creates a local variable that shadows the outer a. So inside test() it prints 10. Outside test(), the global a is still 5, so the second log prints 5.

Q12.

What does this promise chain log?

SNIPPET

```
Promise.resolve(1)
.then(v => v + 1)
.then(v => { throw new Error('oops'); })
.catch(e => console.log(e.message))
.then(() => console.log('done'));
```

A) oops, done
B) 2, oops
C) 1, done
D) oops only

Answer: A The chain: resolve(1) → then adds 1 (v=2) → then throws → catch catches the error and logs 'oops' → the catch returns undefined which resolves, so the next then logs 'done'. Output is: oops then done.

Python

SNIPPET

Q13.

What is the output?

```
def greet(name, greeting="Hello"):
return f"{greeting}, {name}!"
print(greet("Alice"))
print(greet("Bob", "Hi"))
```

A) Hello, Alice! / Hello, Bob!
B) Hello, Alice! / Hi, Bob!
C) Error — missing argument
D) None / None

Answer: B Python supports default argument values. greet('Alice') uses the default greeting='Hello' → 'Hello, Alice!'. greet('Bob', 'Hi') overrides the default → 'Hi, Bob!'.

Q14.

What does this code print?

```
nums = [1, 2, 3, 4, 5]
result = [x**2 for x in nums if x % 2 == 0]
print(result)
```

A) [1, 4, 9, 16, 25]
B) [4, 16]
C) [2, 4]

SNIPPET

D) [1, 9, 25]

Answer: B List comprehension: iterates over nums, keeps only even numbers (2, 4), then squares each → [4, 16]. The condition if x % 2 == 0 filters to even elements before the expression x**2 is applied.

SNIPPET

Q15.

What is the output?

```
d = {'a': 1, 'b': 2, 'c': 3}
print(d.get('d', 0))
```

A) None
B) KeyError
C) 0
D) 'd'

Answer: C dict.get(key, default) returns the value for key if it exists, otherwise returns the default. Since 'd' is not in the dictionary, it returns the provided default value 0.

MCQ

Q16.

Which of the following is TRUE about Python tuples vs lists?

A) Tuples use square brackets; lists use parentheses
B) Tuples are mutable; lists are immutable
C) Tuples are immutable; lists are mutable
D) Tuples cannot store strings

Answer: C Tuples use () and are immutable — you cannot change their elements after creation. Lists use [] and are mutable — elements can be added, removed, or modified. Tuples are generally faster and used for fixed collections of items.

Q17.

What is printed?

SNIPPET

```
class Animal:
def speak(self):
return "..."

class Dog(Animal):
def speak(self):
return "Woof!"

d = Dog()
print(d.speak())
```

A) '...'
B) 'Woof!'
C) AttributeError
D) Both '...' and 'Woof!'

Answer: B This demonstrates method overriding (polymorphism). Dog inherits from Animal but defines its own speak() method. When speak() is called on a Dog instance, Python's MRO finds Dog.speak() first and returns 'Woof!'.

Q18.

SNIPPET

What is the output?

```
def counter():
count = 0
def increment():
nonlocal count
count += 1
return count
return increment

c = counter()
print(c(), c(), c())
```

A) 0 0 0
B) 1 1 1
C) 1 2 3
D) UnboundLocalError

Answer: C This is a closure. nonlocal count lets the inner function modify the enclosing scope's count variable. Each call to c() increments the same count variable, so results are 1, 2, 3.

Q19.

What does this print?

SNIPPET

```
try:
x = int("abc")
except ValueError as e:
print(f"Caught: {e}")
finally:
print("Done")
```

A) Caught: invalid literal... / Done
B) Only 'Done'
C) Only the error message
D) Program crashes

Answer: A int('abc') raises a ValueError because 'abc' cannot be converted. The except block catches it and prints the error message. The finally block ALWAYS runs regardless of exceptions, so 'Done' is also printed.

MCQ

Q20.

What is the purpose of *args and **kwargs in a Python function definition?

A) *args for keyword args; **kwargs for positional args
B) *args collects extra positional args as a tuple; **kwargs collects extra keyword args as a dict
C) They are syntax errors in Python 3
D) *args is for integers only; **kwargs is for strings only

Answer: B *args collects any additional positional arguments into a tuple. **kwargs collects any additional keyword arguments into a dictionary. This lets functions accept variable numbers of arguments.

Q21.

What is the output of this generator code?

```
def gen():
for i in range(3):
yield i * 2
g = gen()
print(next(g), next(g), next(g))
```

A) 0 2 4
B) 0 1 2
C) 2 4 6

SNIPPET

D) StopIteration error

Answer: A yield makes gen() a generator. Each call to next() resumes from where it paused. i goes 0,1,2 → i*2 yields 0, 2, 4. The values are printed separated by spaces.

Q22.

SNIPPET

What is printed?

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

A) [1, 2, 3]
B) [1, 2, 3, 4]
C) [4]
D) Error

Answer: B In Python, lists are mutable objects. b = a does NOT copy the list — both a and b point to the SAME list object in memory. So b.append(4) also modifies what a points to. Use b = a.copy() or b = a[:] to make a copy.

Q23.

What is the result?

```
import re
text = "My phone is 987-654-3210"
match = re.search(r'\d{3}-\d{3}-\d{4}', text)
print(match.group() if match else "No match")
```

A) No match
B) 987-654-3210
C) My phone is
D) Error

Answer: B The regex \d{3}-\d{3}-\d{4} matches exactly 3 digits, hyphen, 3 digits, hyphen, 4 digits — a standard US phone format. re.search() finds it in the string and match.group() returns '987-654-3210'.

Q24.

MCQ

What is a Python decorator?

SNIPPET

A) A special loop that iterates over decorations

B) A function that takes another function and extends its behaviour without modifying itC) A class method that overrides the parent

D) A built-in for formatting strings

Answer: B A decorator is a higher-order function (using @syntax) that wraps another function to add behaviour before/after it runs — without altering the original function's code. Common examples: @staticmethod, @classmethod, @property, @login_required.

Q46.

SNIPPET

What does this code output?

```
data = {"name": "Alice", "age": 30}
for key, value in data.items():
print(f"{key}: {value}")
```

A) Only keys are printed

B) name: Alice / age: 30

C) ['name', 'age']

D) Error — dicts are not iterable

Answer: B dict.items() returns key-value pairs as tuples. Unpacking into key, value in the loop gives both at once. f-string formatting produces 'name: Alice' and 'age: 30' on separate lines.

Selenium (Python)

Q25. SNIPPET

What does the following Selenium code do?

```
from selenium import webdriver
from selenium.webdriver.common.by import By
driver = webdriver.Chrome()
driver.get("https://example.com")
element = driver.find_element(By.ID, "username")
element.send_keys("admin")
```

A) Opens Firefox and searches Google
B) Opens Chrome, navigates to example.com, finds the element with id='username', types 'admin'C) Clicks the element with id='username'
D) Downloads the page source

Answer: B webdriver.Chrome() launches Chrome. driver.get() navigates to the URL. find_element(By.ID, 'username') locates the element whose HTML id is 'username'. send_keys('admin') simulates typing 'admin' into that element.

Q26. MCQ

What is the purpose of WebDriverWait in Selenium?

A) To pause the script for a fixed number of seconds
B) To wait for a specific condition to be true before proceeding (explicit wait)
C) To close the browser after a timeout
D) To handle JavaScript alerts automatically

Answer: B WebDriverWait implements explicit waits — it polls the DOM until a given expected condition (like element_to_be_clickable, visibility_of_element_located) is met or the timeout expires. This is far better than time.sleep() which wastes time and is fragile.

Q27.

What does this Selenium snippet do?

SNIPPET

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By
wait = WebDriverWait(driver, 10)
btn = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[@id='submit']")))
btn.click()
```

A) Waits 10 seconds then always clicks
B) Waits up to 10 sec for the submit button to be clickable, then clicks it
C) Finds all buttons with XPATH and clicks the first one
D) Refreshes the page every 10 seconds

Answer: B WebDriverWait(driver, 10) sets a 10-second timeout. until(EC.element_to_be_clickable(...)) polls until the button is visible and enabled. Once the condition is met (before 10 sec), it returns the element and btn.click() clicks it.

MCQ

Q28.

Which locator strategy should you prefer when elements have unique, stable IDs?

A) By.XPATH with complex expressions
B) By.CSS_SELECTOR with nth-child
C) By.ID
D) By.TAG_NAME

Answer: C By.ID is the fastest and most reliable locator because IDs are unique in HTML and browser engines optimise lookup by ID. XPATH and CSS selectors are more fragile as they depend on document structure.

Q29.

What is the output behaviour of this code?

```
from selenium.webdriver.common.action chains import ActionChains
actions = ActionChains(driver)
actions.move to element(menu).click(submenu).perform()
```

A) It scrolls to the menu then refreshes the page
B) It hovers over menu, clicks submenu, and executes all actions
C) It double-clicks the menu
D) It drags menu to submenu position

Answer: B ActionChains lets you chain complex user interactions. move_to_element(menu) hovers the mouse over the menu element (useful for dropdowns). click(submenu) queues a click on the submenu. perform() executes the entire action chain in sequence.

MCQ

Q30.

How do you handle a browser alert/popup in Selenium Python?

A) driver.find_element(By.ID, 'alert').click()
B) driver.switch_to.alert.accept()
C) driver.close_alert()
D) driver.dismiss()

Answer: B Browser alerts (window.alert, window.confirm) are not part of the DOM. You must switch to them using driver.switch_to.alert which returns an Alert object. Then call .accept() to click OK, or .dismiss() to click Cancel.

SNIPPET

Q31.

What does this snippet accomplish? `driver.execute_script("window.scrollTo(0,`

```
document.body.scrollHeight);")
```

A) Clicks the footer of the page
B) Scrolls the browser window to the bottom of the page
C) Refreshes the page

Q32.

What is the Page Object Model (POM) in Selenium?

A) A method to capture screenshots of each page
B) A design pattern where each web page/component has a corresponding class with its locators andinteractions
C) A Selenium built-in for generating test reports
D) Zooms out the browser

Answer: B execute_script() runs JavaScript in the browser context. window.scrollTo(x, y) scrolls to coordinates (x=0 horizontal, y=document.body.scrollHeight means the maximum vertical scroll, i.e. the bottom). This is handy for lazy-loaded pages.

SNIPPET

D) A way to run tests in parallel

Answer: B POM is a design pattern that creates a class per page (e.g., LoginPage, HomePage). Each class encapsulates locators and actions. This separates test logic from page details, making tests more maintainable and readable.

MCQ

Q47.

What happens if you call driver.quit() vs driver.close() in Selenium?

A) They are identical

B) close() closes only the current tab/window; quit() closes ALL windows and ends the WebDriversession

C) quit() closes only the current tab; close() ends the entire sessionD) close() kills the ChromeDriver process; quit() does not

Answer: B driver.close() closes only the currently focused browser window/tab. driver.quit() shuts down ALL browser windows, kills the WebDriver process, and cleans up the session. Always use quit() at the end of your test to avoid orphan processes.

Robot Framework

Q33.

What does the following Robot Framework test case do?

```
*** Test Cases ***
Login Test
Open Browser https://example.com chrome
Input Text id=username admin
Input Password id=password secret
Click Button id=login-btn
Page Should Contain Welcome
```

A) Opens Firefox and verifies login fails

B) Opens Chrome, fills credentials, clicks login, verifies 'Welcome' appears on pageC) Closes the browser after login

D) Only opens the browser without doing anything

Answer: B Each indented line is a keyword call. Open Browser opens Chrome at the URL. Input Text/Input Password fill the fields. Click Button clicks the login button. Page Should Contain verifies the text 'Welcome' is present in the page source.

Q34. MCQ

In Robot Framework, what is a Resource File?

A) A file containing browser driver binaries

B) A .robot file containing reusable keywords, variables, and settings shared across test suites

C) A CSV file with test data

D) A Python file that installs libraries

Answer: B Resource files (.robot or .resource) are used to store reusable keywords and variables that can be imported into test suites using the Resource setting. They promote DRY (Don't Repeat Yourself) principles in test automation.

SNIPPET

Q35.

What is the purpose of the Setup/Teardown in this Robot Framework suite?

```
*** Settings ***
Suite Setup Open Browser https://example.com chrome
Suite Teardown Close Browser
*** Test Cases ***
Test One
Page Should Contain Home
```

A) Opens a new browser for each test; closes it after each test
B) Opens the browser once before ALL tests in the suite; closes it after ALL tests finishC) Opens the browser only if tests fail
D) Restarts the browser between each test

Answer: B Suite Setup runs ONCE before all test cases in the suite. Suite Teardown runs ONCE after all test cases complete. This is efficient for sharing browser state. Use Test Setup/Teardown (no 'Suite') if you need per-test setup/teardown.

MCQ

Q36.

Which Robot Framework library provides keywords like 'Click Element', 'Input Text', and 'Page Should Contain'?

A) RequestsLibrary
B) SeleniumLibrary
C) BuiltIn
D) DatabaseLibrary

Answer: B SeleniumLibrary is the Robot Framework wrapper around Selenium WebDriver. It provides browser-interaction keywords. BuiltIn provides general keywords (Log, Should Be Equal). RequestsLibrary is for API testing.

Q37.

What does this variable syntax mean in Robot Framework?

SNIPPET

```
*** Variables ***
${BASE_URL} https://example.com
@{BROWSERS} chrome firefox edge
```

A) ${} is a list variable; @{} is a scalar variable
B) ${} is a scalar (single value); @{} is a list variable
C) Both are the same type
D) @{} is for integers only

Answer: B In Robot Framework: ${var} is a scalar variable holding a single value. @{var} is a list variable. &{var} is a dictionary variable. So ${BASE_URL} holds one URL string, while @{BROWSERS} holds a list of three browser names.

Q38.

SNIPPET

What does this custom keyword do?

```
*** Keywords ***
Login As User
[Arguments] ${username} ${password}
Input Text id=user ${username}
Input Password id=pass ${password}
Click Button id=login
```

A) It defines a test case named 'Login As User'
B) It creates a reusable keyword that accepts username and password and performs login steps
C) It imports a library called 'Login As User'
D) It sets global variables

Answer: B This defines a User Keyword in the *** Keywords *** section. [Arguments] declares the parameters it accepts. The keyword encapsulates three steps. It can be called from any test case as: Login As User admin secret123.

Q49.

SNIPPET

What does [Documentation] do in this Robot Framework test?

```
*** Test Cases ***
Verify Login
[Documentation] Verify that a valid user can log in successfully.
[Tags] smoke regression
Open Browser https://app.com chrome
Input Text id=user admin
```

A) [Documentation] runs as a test step
B) [Documentation] adds a description to the test visible in reports; [Tags] categorise the test
C) [Documentation] sets a variable called 'Documentation'
D) It causes the test to be skipped

Answer: B [Documentation] is a test setting that adds human-readable description visible in HTML reports and logs — it does not execute as a step. [Tags] categorise tests so you can run subsets with --include smoke.

API Testing

Q39.

SNIPPET

What does this Python requests snippet do and what is it verifying?

```
import requests
response = requests.get("https://jsonplaceholder.typicode.com/posts/1")
assert response.status_code == 200
assert response.json()['id'] == 1
```

A) Sends a POST request and checks the response body
B) Sends a GET request, asserts HTTP 200, and checks that the JSON 'id' field equals 1
C) Deletes a post with id=1

D) Logs in to jsonplaceholder

Answer: B requests.get() sends an HTTP GET request. status_code == 200 asserts a successful response. response.json() parses the JSON body into a dict. ['id'] == 1 checks that the id field in the response is 1. This is a basic API assertion test.

MCQ

Q40.

What HTTP method should be used to UPDATE an existing resource partially?

A) PUT
B) POST
C) PATCH
D) DELETE

Answer: C PATCH is used for partial updates (only send fields you want to change). PUT replaces the entire resource. POST creates a new resource. DELETE removes a resource. This is a common REST API interview question.

SNIPPET

Q41.

What does this snippet test?

```
import requests

payload = {"title": "Test", "body": "Content", "userId": 1}

headers = {"Content-Type": "application/json"}

response = requests.post(

"https://jsonplaceholder.typicode.com/posts",

json=payload,

headers=headers

)

print(response.status_code)
```

A) Sends a GET request with JSON body
B) Sends a POST request with JSON payload and prints the response status code
C) Updates an existing post
D) Deletes a post

Answer: B requests.post() sends a POST request. json=payload auto-serialises the dict to JSON and sets Content-Type:application/json. The response status is printed — for a successful creation, REST APIs typically return 201 Created.

MCQ

Q42.

What is the difference between HTTP status codes 401 and 403?

A) 401 = Not Found; 403 = Server Error

B) 401 = Unauthenticated (no/invalid credentials); 403 = Forbidden (authenticated but no permission)C) 401 = Success; 403 = Redirect

D) They are identical

Answer: B 401 Unauthorized means the request lacks valid authentication credentials — the client should log in. 403 Forbidden means the client IS authenticated but does NOT have permission to access the resource.

SNIPPET

Q43.

What does this Robot Framework API test verify?

```
*** Settings ***
Library RequestsLibrary
*** Test Cases ***
Verify Get Post
Create Session api https://jsonplaceholder.typicode.com
${resp}= GET On Session api /posts/1
Should Be Equal As Integers ${resp.status code} 200
Dictionary Should Contain Key ${resp.json()} title
```

A) Sends a POST request and checks for a 201 response

B) Sends a GET request to /posts/1, checks HTTP 200, and verifies 'title' key exists in JSON

C) Deletes post 1 and checks deletion

D) Creates a new session and closes it

Answer: B Create Session creates a named HTTP session. GET On Session sends the GET request. Should Be Equal As Integers checks status 200. Dictionary Should Contain Key verifies the response JSON has a 'title' key — a typical API response validation.

MCQ

Q44.

What is the purpose of an API Authentication Token (Bearer Token)?

A) To encrypt the request body
B) To identify and authorise the caller without sending username/password in each requestC) To specify the API version
D) To cache API responses

Answer: B Bearer tokens (JWT or OAuth tokens) are sent in the Authorization header. They allow the server to identify and authorise the requester without needing credentials on every call. Once authenticated, you get a token; subsequent requests use the token.

SNIPPET

Q45.

What does this snippet test?

```
import requests

response = requests.delete(
"https://jsonplaceholder.typicode.com/posts/1"
)
assert response.status_code == 200
print("Deleted successfully")
```

A) Creates a new post with id=1
B) Sends an HTTP DELETE request for post 1 and asserts the deletion was acknowledged
C) Retrieves and prints post 1
D) Updates post 1

Answer: B requests.delete() sends an HTTP DELETE request. A 200 OK (or 204 No Content) response means the server acknowledged the deletion. Asserting the status code is the standard way to verify a DELETE API call succeeded.

Q48.

What does this code snippet check?

```
import requests, json

response = requests.get("https://jsonplaceholder.typicode.com/posts")

data = response.json()

assert isinstance(data, list)

assert len(data) > 0

print(f"Total posts: {len(data)}")
```

A) It checks the response is a JSON object (dict) with at least 1 key

B) It checks the response is a JSON array with at least one item and prints the countC) It checks the first post's title

D) It sends a POST and verifies creation

Answer: B response.json() parses the response. isinstance(data, list) asserts it's a JSON array (Python list). len(data) > 0 ensures the array is not empty. This is a common schema-level validation in API testing.

Python / API

Q50.

What does this complete API test function do?

```
import requests
import pytest
BASE = "https://jsonplaceholder.typicode.com"
def test_create_and_verify_post():
payload = {"title": "Buy milk", "body": "2 litres", "userId": 1}
r = requests.post(f"{BASE}/posts", json=payload)
assert r.status_code == 201
body = r.json()
assert body["title"] == "Buy milk"
assert "id" in body
```

A) It sends a GET and checks 200
B) It sends a POST to create a post, asserts 201 status, and validates the response body containscorrect title and an id
C) It updates an existing post and checks 200
D) It only checks that the server is reachable

Answer: B This is a complete pytest API test. It POSTs a new resource, checks 201 Created (standard for resource creation), then validates the response JSON has the correct title and that the server assigned an id. This pattern — send → assert status → assert body — is the foundation of API testing.